University for the Creative Arts

BERLIN SCHOOL OF BUSINESS & INNOVATION

Essay / Assignment Title: **Online Marketplace Data Warehouse Management & Business Analytics**

Programme title: **MSc. DATA ANALYTICS**

Name: **SEFA ÜNVEREN**

Year: **2025**

CONTENTS

Cover 1

Contents 2

Statement of compliance with academic ethics and the avoidance of plagiarism 3

Introduction 4

Chapter 1 / Task 1 5

Chapter 2 / Task 2 13

Chapter 3 / Task 3 30

Concluding Remarks 33

Bibliography 34

Appendix 35

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname (Capital letters):

.....SEFA ÜNVEREN.....

Date: ....12..../..02../…2025...

INTRODUCTION

In today's competitive commercial world, making data-driven decisions is crucial for companies. E-commerce platforms bring together customers and vendors and produce huge amounts of data as a result. Effectively collecting, processing, and analyzing this data plays a crucial role in developing business strategies and customer experience.

I have been hired to design and manage an analytics data warehouse for a prominent global online shopping marketplace that connects customers and vendors. This data warehouse should support business analytics related to the company's popular online shopping portal.

Our marketplace has signed the same agreement with all vendors. According to this agreement, the marketplace charges no commission on sales instead, vendors must pay 50000 $ annual fee. But there are some strict rules that all vendors must obey. Our marketplace has a one-price policy for all products within the same category. Currently, we are operating on a trial basis with 5 category types and 5 products. If this trial period succeeds for everyone (vendors, customers, or delivery companies), we will upgrade our system, expand product variety, and allow free pricing and open-market policies.

Our marketplace has a principle for the personal information protection policy. Therefore our database is not recording or tracing incomplete orders. However, in our data warehouse, we only store orders that have been completed and delivered to customers but some of them have not been delivered yet due to some reasons such as delivery delays, address errors, etc.

All vendors are strictly required to update their stock status in real time as our online marketplace has a zero-tolerance policy for out-of-stock products. Every product must be available in stock and stock tracking is on vendor's responsibility. All vendors are obligated to guarantee that enough products is on stock.

CHAPTER ONE

TASK 1 – ANALYTIC DATA WAREHOUSE DESIGN

SUB-TASK 1.1

My first objective will focus on the vendor's revenue and best–selling products. These relations are crucial for the online shopping marketplace.

'Which vendors are getting the most revenue?' and 'Which products are the best-selling?'

With this objective, I want to analyze the most popular products and vendors that get the most revenue. As a result of this analysis, I can understand which products and vendors are earning the most profit on our platform.

The second objective will focus on the customer's purchasing behaviors and product reviews and their influence on the shopping experience.

'What are customers' purchasing behaviors 'and 'How do product reviews affect sales? '

With this objective, I want to analyze customers' purchasing behaviors and the effect of product reviews and ratings on sales. As a result of this analysis, I can understand customers' purchasing behaviors and the impact of product reviews on their decisions.

The third objective will focus on the customer's age and customers' satisfaction or customers' age effects on reviews.

'How does a customer's age affect their satisfaction' and 'How does a customer's age affect their review in a good or bad way?'

With this objective, I want to analyze the effect of customer age on shopping satisfaction and the effect on the customer's reviews.

SUB-TASK 1.2

My dataset's Entities as shown below:

- **Customer:** This entity represents customers who make shopping from our online marketplace. This entity includes attributes such as customer ID, name, e-mail, and address. Customer ID is our primary key.
- **Comments:** This entity represents comments made by customers about products. This entity includes attributes such as comment ID, customer ID, name, e-mail, and comments. Comment ID is our primary key and customer ID is our foreign key linking comments to customers.

- **Reviews:** This entity represents reviews made by customers about products. This entity includes attributes such as review ID, customer ID, review, and ratings. Review ID is our primary key. Customer ID is our foreign key in this entity.

- **Order:** This entity represents orders that are given by a customer for one or more products. This entity includes attributes such as order ID, order date, customer ID, total amount, and order status. Order ID is our primary key and customer ID is our foreign key linking orders to customers.

- **Delivery:** This entity represents delivery for orders that are given by a customer for one or more products. This entity includes attributes such as delivery ID, order ID, delivery status, customer name, shipment date, and delivery date. Delivery ID is our primary key and Order ID is our foreign key linking deliveries to orders.

- **Order Details:** This entity represents order details for each order that is given by customers. This entity includes attributes such as order details ID, order ID, product ID, quantity, unit price, and total price. Order details ID is our primary key and order ID and product ID are our foreign keys in this entity.

- **Products:** This entity represents products that are sold by vendors. This entity includes attributes such as product ID, vendor ID, product name, category ID, and purchase price. Products ID is our primary key and vendor ID and category ID are our foreign keys.

- **Vendor:** This entity represents vendors who make selling products in our online marketplace. This entity includes attributes such as vendor ID, name, and Address. Vendor ID is our primary key.

- **Categories:** This entity represents categories that all products include. This entity includes attributes such as category ID, category name, and category type. Category ID is our primary key.

Every entity has a relationship with another entity. Now let's check deeply these relationships.

- **Writes:** This relationship is between customers and comments. A customer can write many comments but each comment is written by only one customer. This is a one-to-many relationship.

- **Have:** This relationship is between customers and reviews. A customer can write many reviews but each review is written by only one customer. This is a one-to-many relationship.

- **Give:** This relationship is between customers and orders. A customer can give many orders but each order is given by only one customer. This is a one-to-many relationship.

- **Shipment:** This relationship is between orders and delivery. An order can place many deliveries but each delivery has one order. This is a one-to-many relationship.

- **Contains:** This relationship is between orders and order details. An order can contain many order details but each order detail belongs to only one order. This is a one-to-many relationship.
- **Includes:** This relationship is between products and order details. A product can appear in many order details but each order detail belongs to only one product. This is a one-to-many relationship.
- **Belongs To:** This relationship is between products and categories. A category can contain many products but each product belongs to only one category. This is a one-to-many relationship.
- **Selling:** This relationship is between vendors and products. A vendor can sell many products but each product belongs to only one vendor. This is a one-to-many relationship.

ER Diagram is shown below and drawn with www.miro.com

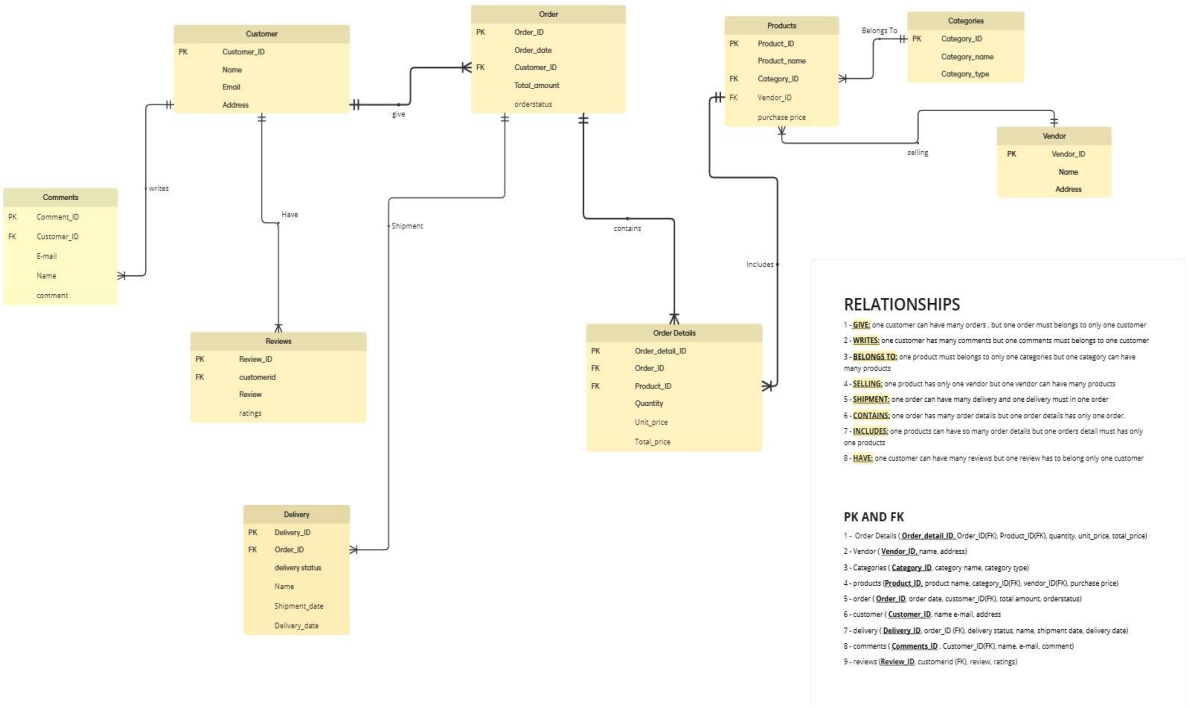

Figure 1 – ER diagram drawn with www.miro.com

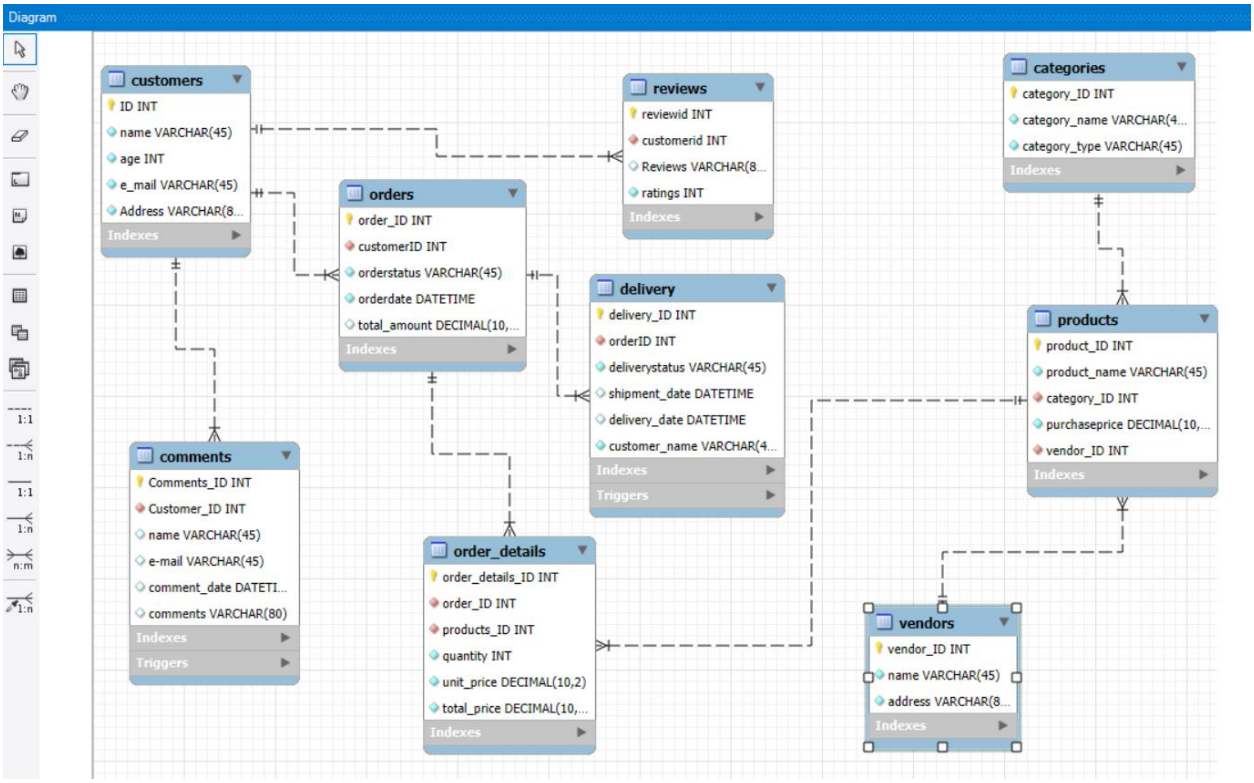

Figure 2 – ER diagram prepared with MYSQL Workbench

SUB-TASK 1.3

After drawing the ER diagram, I will write fully normalized tables and entities:

- Order Details (Order_details_ID (PK), Order_ID(FK), Product_ID(FK), quantity, unit_price, total_price)
- Vendor (Vendor_ID(PK), name, address)
- Categories (Category_ID(PK), category name, category type)
- Products (Product_ID(PK), product name, category_ID(FK), vendor_ID(FK), purchase price)
- Order (Order_ID(PK), order date, customer_ID(FK), total amount, order status)
- Customer (Customer_ID(PK), name e-mail, address)
- Delivery (Delivery_ID(PK), order_ID (FK), delivery status, name, shipment date, delivery date)
- Comments (Comments_ID(PK) , Customer_ID(FK), name, e-mail, comment)
- Reviews (Review_ID(PK), customerid (FK), review, ratings)

SUB-TASK 1.4

This database is designed to support an online marketplace. Detailed explanations of the design choices for each entity and its attributes are provided below.

In the customer entity; includes customer information such as customer ID, name, e-mail, and address. Customer ID is the primary key as mentioned above in sub-task 1.2. other attributes are included to support customer communication and authentication.

In the delivery entity; includes delivery information such as such as delivery ID, order ID, delivery status, customer name, shipment date, and delivery date. Delivery ID is our primary key and Order ID is our foreign key linking deliveries to orders as mentioned above in sub-task 1.2. Delivery status indicates whether an order has been delivered to the customer or not delivered yet. The shipment date indicates when the order was shipped and the delivery date reflects when it was delivered to the customer. If the order status has not been delivered yet, the delivery date remains null.

In the comments entity; includes comments information such as comment ID, customer ID, name, e-mail, and comments. Comment ID is our primary key and customer ID is our foreign key linking comments to customers as mentioned above in sub-task 1.2. The name and e-mail are included to support customer communication and authentication. The comments store customer's feedback about products and services.

In the reviews entity; includes reviews and ratings such as review ID, customer ID, review, and ratings. Review ID is our primary key. Customer ID is our foreign key in this entity as mentioned above in sub-task 1.2. The review stores comments and ratings submitted by customers as part of their feedback.

In the order entity; includes order information such as order ID, order date, customer ID, total amount, and order status. Order ID is our primary key and customer ID is our foreign key linking orders to customers as mentioned above in sub-task 1.2. Order date indicates the date when the order was placed, and the total amount refers to the total purchase for each order. Order status indicates the current stage of an order which aids in tracking the orders.

In the order details entity; includes all information about the orders such as order details ID, order ID, product ID, quantity, unit price, and total price. Order details ID is our primary key and order ID and product ID are our foreign keys linking order details to orders and products as mentioned above in sub-task 1.2. Quantity represents the number of products which is ordered by customers and unit price indicates the selling price of each product. The total price is equal to quantity * unit price. The total price has the same value as the total amount in the order entity.

In the products entity; includes all information about products such as product ID, vendor ID, product name, category ID, and purchase price. The product ID is our primary key and vendor ID and category ID are our foreign keys linking products to vendors and categories as mentioned above in sub-task 1.2. the

Product name refers to the name of the product and the purchase price indicates the vendor's buying cost.

In the vendor entity; includes all information related to vendors such as vendor ID, name, and Address. Vendor ID is our primary key as mentioned above in sub-task 1.2. Name and address are included to support vendor authentication.

In the categories entity; includes detailed information that aids departing products such as category ID, category name, and category type. Category ID is our primary key as mentioned above in sub-task 1.2. The category name shows which products are included, and the category type sorts them into specific groups.

SUB-TASK 1.5

Now according to our entities and their attributes, we can implement them into the MySQL database management system.

After opening the MySQL workbench program and entering my username and password, I started by creating a schema in the navigator menu on the left side of the screen. After giving the name, I clicked the apply button to create a new schema.

After creating the 'assignment' schema, I started to create tables into schema based on entities and their attributes.

- **Order Details:**

  Order Details (Order_details_ID (Primary Key, auto incremental, not null and datatype = INT)

  Order_ID(Foreign Key, not null and datatype = INT)

  Product_ID(Foreign Key, not null and datatype = INT)

  Quantity(not null and datatype = INT)

  Unit_price and Total_price(not null and datatype = DECIMAL(10,2))

- **Vendors:**

  Vendor (Vendor_ID(Primary Key, auto incremental, not null and datatype = INT)

  Name(not null, datatype= VARCHAR(45))

  Address(not null, datatype= VARCHAR(80)))

- **Categories:**

  Categories (Category_ID(Primary Key, auto incremental,not null and datatype = INT)

Category name(not null, datatype= VARCHAR(45))

Category type(not null, datatype= VARCHAR(45)))

- **Products:**

  Products (Product_ID(Primary Key, auto incremental,not null and datatype = INT)

  Product Name(not null, datatype= VARCHAR(45))

  Category_ID(Foreign Key, not null and datatype = INT)

  Vendor_ID(Foreign Key, not null and datatype = INT)

  Purchase price(not null and datatype = DECIMAL(10,2))

- **Orders:**

  Order (Order_ID(Primary Key, auto incremental,not null and datatype = INT)

  Order date(not null, datatype= DATETIME)

  customer_ID(Foreign Key, not null and datatype = INT)

  Total amount (not null and datatype = DECIMAL(10,2))

  order status(not null, datatype= VARCHAR(45))

- **Customers:**

  Customer (Customer_ID(Primary Key, auto incremental,not null and datatype = INT)

  Name(not null, datatype= VARCHAR(45))

  e-mail(not null, datatype= VARCHAR(45))

  Address(not null, datatype= VARCHAR(80))

  Age(not null and datatype = INT)

- **Delivery:**

  Delivery (Delivery_ID(Primary Key, auto incremental,not null and datatype = INT)

  Order_ID (Foreign Key, not null and datatype = INT)

  delivery status(not null, datatype= VARCHAR(45))

  name(not null, datatype= VARCHAR(45))

shipment date and delivery date(datatype= DATETIME))

- **Comments:**

  Comments (Comments_ID(Primary Key, auto incremental,not null and datatype = INT)

  Customer_ID(Foreign Key, not null and datatype = INT)

  Name(not null, datatype= VARCHAR(45))

  e-mail (datatype= VARCHAR(45))

  comment (datatype= VARCHAR(80))

- **Reviews:**

  Reviews (Review_ID(Primary Key, auto incremental,not null and datatype = INT)

  Customerid (Foreign Key, not null and datatype = INT)

  Review(datatype= VARCHAR(80))

  Ratings(not null and datatype = INT))

I paid special attention to the price or amount attributes, choosing DECIMAL(10,2) as their datatype for these attributes to handle large values and maintain decimal precision.

Another important point is related to the address attributes. Initially, I chose VARCHAR(45) as the data type, but I received an error indicating that the value was not sufficient so I had 2 options: either shorten the address information or increase the value. Then I decided to increase the value to VARCHAR(80). This value had enough character for the address attributes.

I selected the DATETIME datatype for fields like delivery date or shipment date.

CHAPTER TWO

TASK 2 – DATA ANALYSIS WITH SQL

SUB-TASK 2.1

In this part, I need to add enough additional information to my dataset. I decided to add enough information by using Python codes. I am using Python on Google Colab but Google Colab is a cloud-based Python program that cannot directly connect to MySQL Workbench. To achieve this; I had two options: either make MySQL Workbench dataset publicly accessible or use a program that builds a secure tunnel between MySQL Workbench and Google Colab. If I chose the first option and opened my dataset to public access; My personal information could be exposed and this option is totally insecure but the second option is more reliable than the first option.

To create a secure tunnel, I used the 'Ngrok' program which is free to access and download easily. After the download, I followed the installation steps. After installation, I opened Windows Command Prompt as an administrator and Started typing the codes as shown below. The code was 'ngrok authtoken 2sWxnZMBW5FT3xeQwMoyIbIKGfZ_6758FQM1Sb5AABFNymfR'

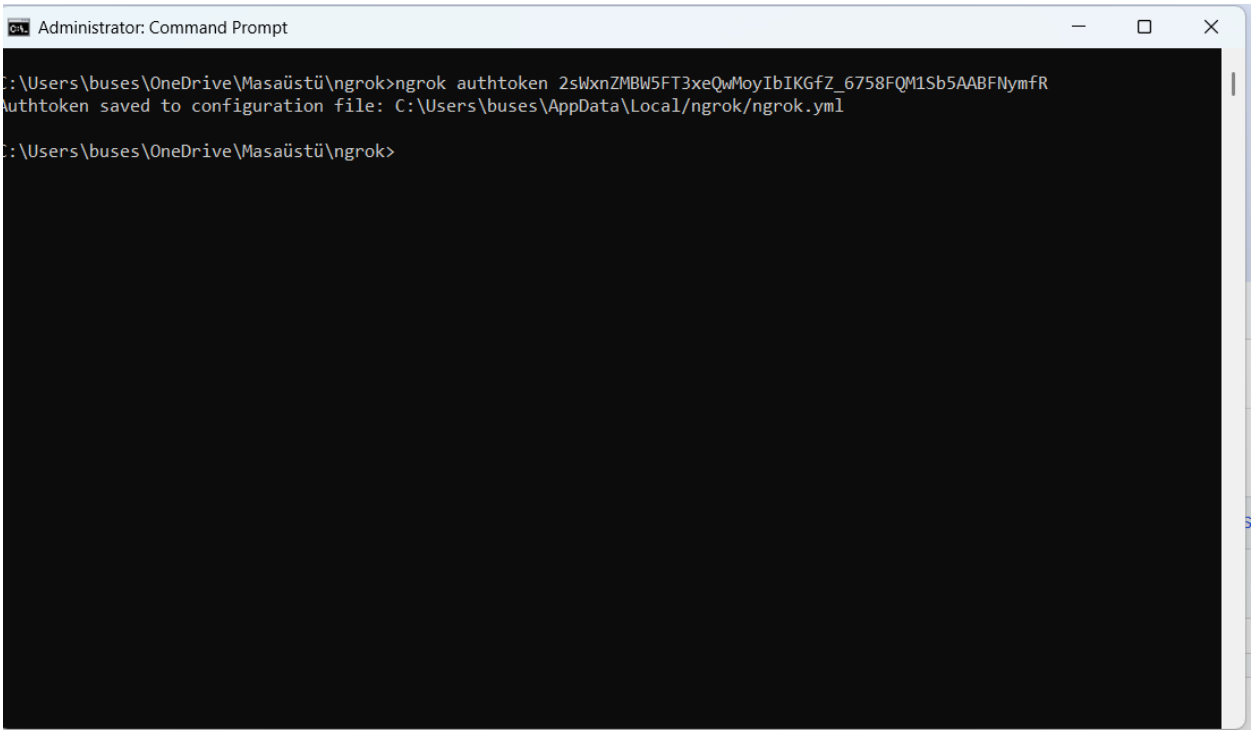

Figure 3 – Ngrok Connection with Personal Authtoken

The next step is creating a tunnel using MySQL port when I wrote ‘ngrok tcp 3306’ the secure tunnel was created.

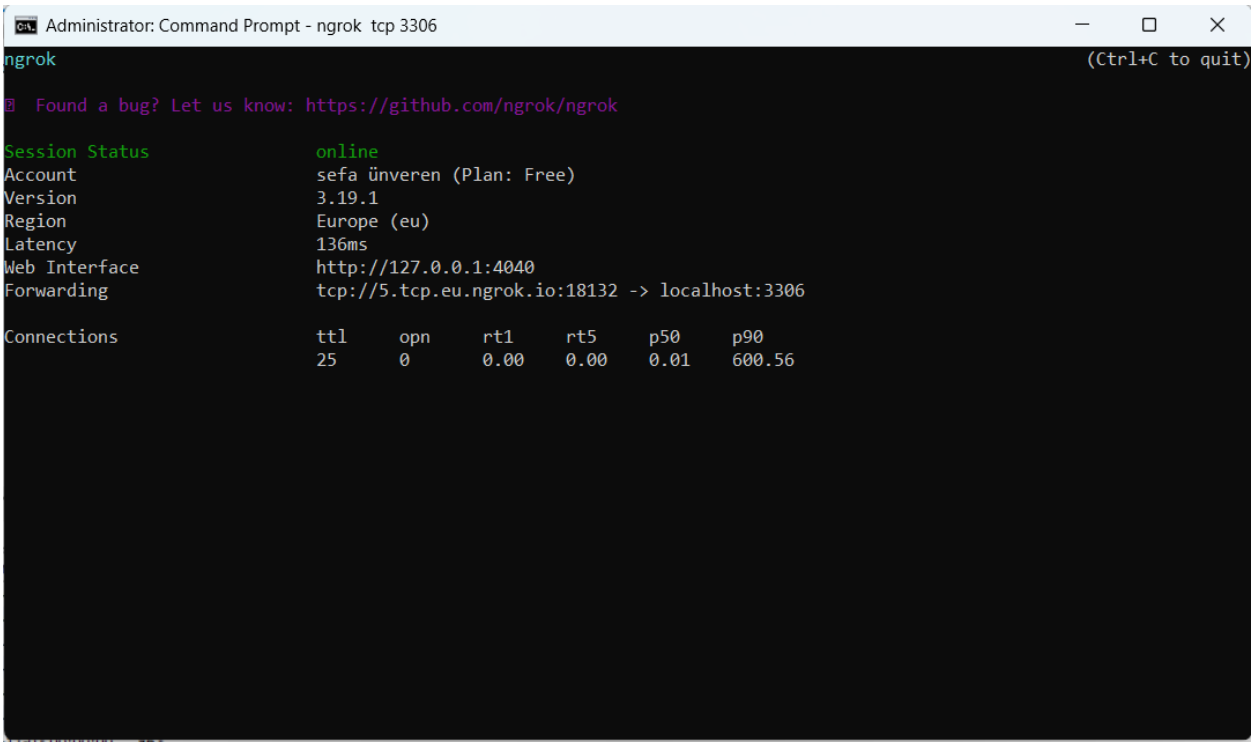

Figure 4 – Windows Command Prompt with Ngrok

Now I can connect Google Colab with MySQL Workbench using this secure tunnel. I opened google colab from my internet browser. My first job was to write the necessary codes for the MySQL and Python connections.

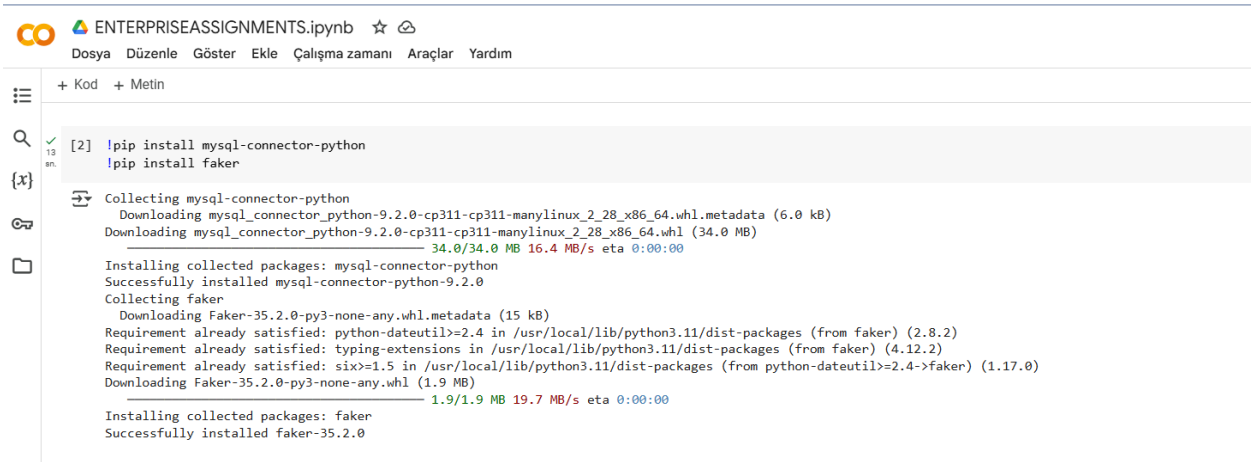

Figure 5 – Python Codes for the MySQL and Python connector

Now it's time to load necessary libraries for Python.

```
[3]  import mysql.connector
     from faker import Faker
     from datetime import datetime, timedelta
     import random
```

```
[4]  fake = Faker ()
```

Figure 6 – Necessary Libraries

The Faker library is used to generate fake information for other platforms. Now I can write the necessary information for making connections between MySQL workbench and Google Colab. When I ran the codes, the connection was successful.

```
db = mysql.connector.connect(
    host = '5.tcp.eu.ngrok.io',
    port = 18132,
    user = 'root',
    password = 'Sefa1986.',
    database = 'assignment',
       auth_plugin='mysql_native_password'
)

cursor = db.cursor()
cursor.execute('SHOW DATABASES;')
for x in cursor:
    print(x)
```

```
('assignment',)
('bsbibank',)
('ilkdatabase',)
('information_schema',)
('mysocialmedia',)
('mysql',)
('performance_schema',)
('pizzahot',)
('sakila',)
('socialmedia',)
('stockexchangedmarket',)
('sys',)
('world',)
```

Figure 7 – Google Colab and MySQL Connections

In this code; I used a direct host address which is given by ngrok in figure 4. That's the way of a secure tunnel. Port is also given by ngrok which is shown in figure 4.

Now I can generate fake information using Google Colab Python codes.

I preferred to generate fake information for every single table separately so that I could notice and correct any mistake in the early stages. I started with categories tables by writing codes which are shown in figure 8.

```
[ ] categories = [('mobile phone', 'electronics'), ('sofa', 'furniture'), ('ring', 'accessories'), ('t-shirt', 'clothing'), ('running shoes', 'shoes')]

for _ in range(300):
    category = random.choice(categories)
    category_name, category_type = category

    query = 'INSERT INTO categories (category_name, category_type) VALUES (%s, %s)'
    values = (category_name, category_type)
    cursor.execute(query, values)
    db.commit()
```

Figure 8 – Writing Python Code for the Categories tables

With this code I defined categories and I wanted to make selections in these specific categories and products. Based on this code, I managed to add 300 pieces of information in categories tables.

```
for _ in range(300):
    name = fake.name()
    e_mail = fake.email()
    address = fake.address()
    age = random.randint(18, 50)

    query = 'INSERT INTO customers (name, e_mail, address, age) VALUES (%s, %s, %s, %s)'
    values = (name, e_mail, address, age)
    cursor.execute(query, values)
    db.commit()
```

Figure 9 - Writing Python Code for the Customer's tables

Based on the codes which are shown in figure 9, I managed to add 300 pieces of information including fake names, fake addresses, fake e-mails, and fake ages between 18-50. I preferred age limits between 18-50 because according to my observations, people who are aged between 18-50, are shopping online more than older people.

```
[14] cursor.execute('SELECT ID, name, e_mail FROM customers')
     customers = cursor.fetchall()
     comments_list = ['Amazing product!','I loved it!','Very high quality.','I definitely do not recommend.','Very poor quality product.']
     for _ in range(300):
         customer = random.choice(customers)
         customer_id, name, e_mail = customer
         comments = random.choice(comments_list)
         comment_date = fake.date_time_between(start_date='-1y', end_date=datetime.today())

         query = 'INSERT INTO comments (customer_id, name, `e-mail`, comment_date, comments) VALUES (%s, %s, %s, %s, %s)'
         values = (customer_id, name, e_mail, comment_date, comments)
         cursor.execute(query, values)
         db.commit()
```

Figure 10 – Writing Python Code for the Comments tables

Adding fake information in the comments table is a little bit harder than previous ones. Because in comments tables, I have to pay attention to customer's information. Because customer names and e-mails have to be taken from customers' tables, this information can not be mixed. I defined comments, and I wanted to make selections in these specific comments. In this code, the comment date generates fakes and limits between today and 1 year before. But it was a mistake because, in the real world, a customer can't comment before receiving products or, more clearly, before the delivery date. To correct this mistake, I wrote SQL queries in MySQL Workbench, as shown below.

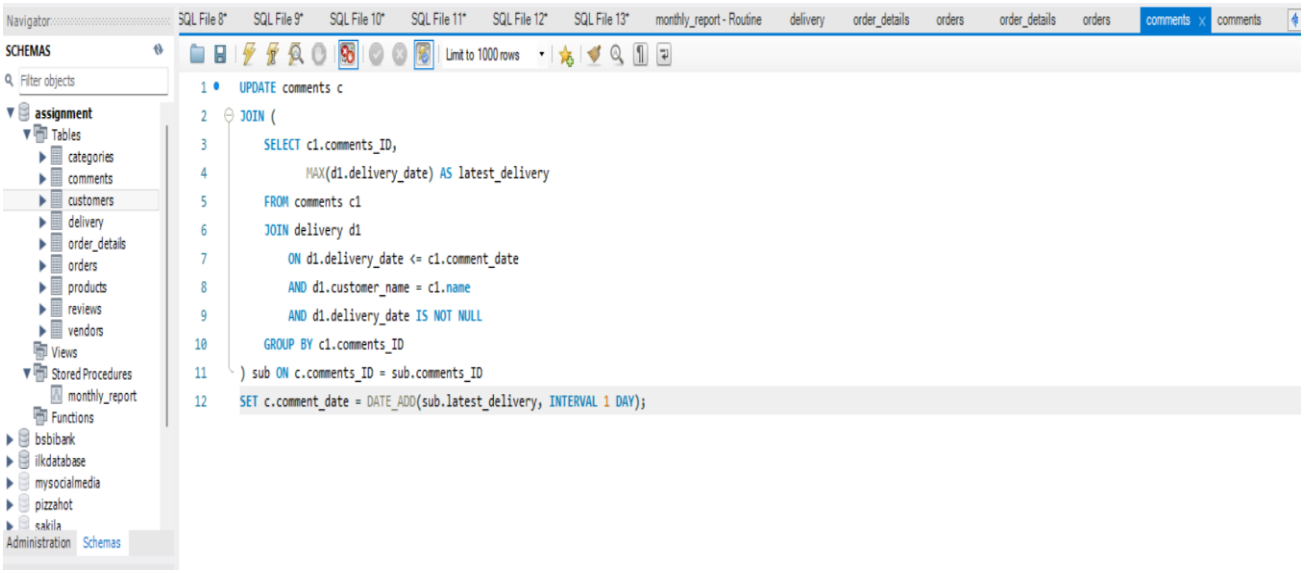

Figure 11 – Writing SQL Queries for the date mistake

```python
for _ in range(300):
    name = fake.name()
    address = fake.address()

    query = 'INSERT INTO vendors (name, address) VALUES (%s, %s)'
    values = (name, address)
    cursor.execute(query, values)
    db.commit()
```

Figure 12 - Writing Python Code for the Vendors' Tables

Based on this code, I managed to add 300 information vendor tables successfully. With this code, it generates 300 fake names and fake addresses for vendor tables.

```python
cursor.execute('SELECT ID FROM customers')
customer_ids = [row[0] for row in cursor.fetchall()]
for _ in range(300):
    customerid = random.choice(customer_ids)
    orderstatus ='completed'
    orderdate = fake.date_between(start_date='-1y', end_date='today')
    total_amount = round(random.uniform(100, 5000), 2)

    query = 'INSERT INTO orders (customerid, orderstatus, orderdate, total_amount) VALUES (%s, %s, %s, %s)'
    values = (customerid, orderstatus, orderdate, total_amount)
    cursor.execute(query, values)
    db.commit()
```

Figure 13 - Writing Python Code for the Orders tables

Based on this code, I managed to add 300 information order tables successfully. With this code, it generates 300 fake dates for one-year intervals and generates orders for customers who are chosen from customers' tables. Also, in the same way, this code generates 300 random numbers for the total amount, but unfortunately, this is a mistake because the total amount must be equal to the total price in the order details tables. To correct this mistake, I wrote SQL queries in MySQL Workbench, as shown below.

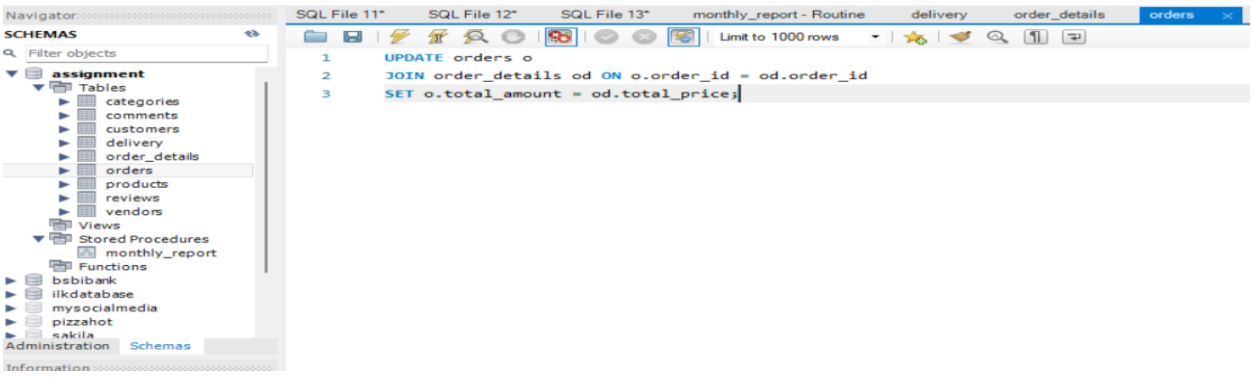

Figure 14 - Writing SQL Queries for the total amount mistake

```
cursor.execute('SELECT vendor_ID FROM vendors')
vendor_ids = [row[0] for row in cursor.fetchall()]
cursor.execute('SELECT category_ID, category_type FROM categories')
category_data = cursor.fetchall()
category_map = {category_type: category_ID for category_ID, category_type in category_data}
price_list = [('mobile phone', 1500.00, 'electronics'), ('sofa', 500.00, 'furniture'), ('ring', 2.60, 'accessories'), ('t-shirt', 2.00, 'clothing'), ('running shoes', 30.00, 'shoes')]

for _ in range(300):
    vendorid = random.choice(vendor_ids)
    product_name, price, category_type = random.choice(price_list)
    categoryid = category_map.get(category_type, random.choice(list(category_map.values())))

    query = 'INSERT INTO products (vendor_id, category_id, price, product_name) VALUES (%s, %s, %s, %s)'
    values = (vendorid, categoryid, price, product_name)
    cursor.execute(query, values)
    db.commit()
```

Figure 15 - Writing Python Code for the product tables

This code generates 300 random products in the products tables. I defined a price list for each product and category so that It ensures that each product has an associated vendor, category, price, and product name.

```
cursor.execute('SELECT order_ID FROM orders')
orderids = [row[0] for row in cursor.fetchall()]
cursor.execute('SELECT name FROM customers')
customer_names = [row[0] for row in cursor.fetchall()]
values_to_insert = []

for _ in range(300):
    orderid = random.choice(orderids)
    customer_name = random.choice(customer_names)

    shipment_date = fake.date_time_between(start_date='-1y', end_date=datetime.today())

    delivery_date = shipment_date + timedelta(days=random.randint(1, 3))

    deliverystatus = random.choice(['delivered', 'not delivered yet'])

    values_to_insert.append((orderid, customer_name, shipment_date, delivery_date, deliverystatus))
    query = 'INSERT INTO delivery (orderid, customer_name, shipment_date, delivery_date, deliverystatus) VALUES (%s, %s, %s, %s, %s)'
    cursor.executemany(query, values_to_insert)
    db.commit()
```

Figure 16 - Writing Python Code for the delivery tables

This code generates 300 random delivery records in the delivery tables. It ensures that each delivery is linked to an order ID, customer name, shipment date, delivery date, and delivery status. With this code, it generates 300 fake dates to shipment dates for one-year intervals, and it also generates fake delivery dates 1-3 days intervals after shipment date. I defined delivery status with two options: delivered or not delivered yet. There is a mistake here if the delivery status is not delivered yet, the delivery date also must be null. To correct this mistake, I wrote SQL queries in MySQL Workbench, as shown below.

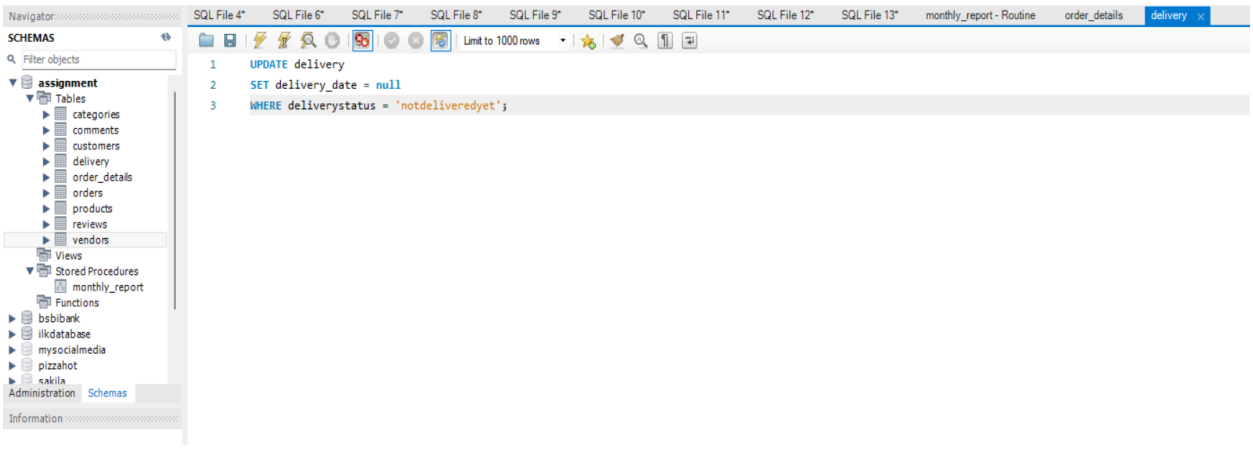

Figure 17 - Writing SQL Queries for the delivery date mistake

```
cursor.execute('SELECT order_ID FROM orders')
orderids = [row[0] for row in cursor.fetchall()]
cursor.execute('SELECT product_ID, price FROM products')
product_prices = {row[0]: row[1] for row in cursor.fetchall()}
values_to_insert = []

for _ in range(300):
    orderid = random.choice(orderids)
    productid = random.choice(list(product_prices.keys()))
    quantity = random.randint(1, 10)
    unit_price = product_prices[productid]
    total_price = quantity * unit_price

    values_to_insert.append((orderid, productid, quantity, unit_price, total_price))

query = 'INSERT INTO order_details (order_ID, products_ID, quantity, unit_price, total_price) VALUES (%s, %s, %s, %s, %s)'
cursor.executemany(query, values_to_insert)

db.commit()
```

Figure 18 - Writing Python Code for the order details tables

This code generates 300 order details records in the order details tables. It ensures that each order detail is linked to an order, product, quantity, unit price, and total price. With this code, it generates 300 fake numbers in 1 – 10 intervals for the quantity. With this code, it selects an order ID from the orders tables and a product ID with its price from the products tables. In this code, it calculates the total price by multiplying the quantity by the unit price.

```
[27] cursor.execute('SELECT customer_ID, comments from comments')
     comments = cursor.fetchall()
     comment_rating_map = {'Amazing product!': 5,'I loved it!': 4,'Very high quality.': 3,'I definitely do not recommend.': 2,'Very poor quality product.': 1}
     for customerid, comment in comments:
         rating = comment_rating_map.get(comment)

         if rating is not None:
             query = 'INSERT INTO reviews (customerid, reviews, ratings) VALUES (%s, %s, %s)'
             values = (customerid, comment, rating)
             cursor.execute(query, values)
             db.commit()
```

Figure 19 - Writing Python Code for the review tables

This code maps specific customer comments to numerical ratings and inserts them into a reviews table. I defined rating from 1 to 5 according to comments.

SUB-TASK 2.2

Now that the dataset is prepared with sample data, we can execute SQL queries to explore it. For sub-task 1.1, I focused on identifying the best-selling products and wrote the following SQL query to answer this question, as shown in figure 20.

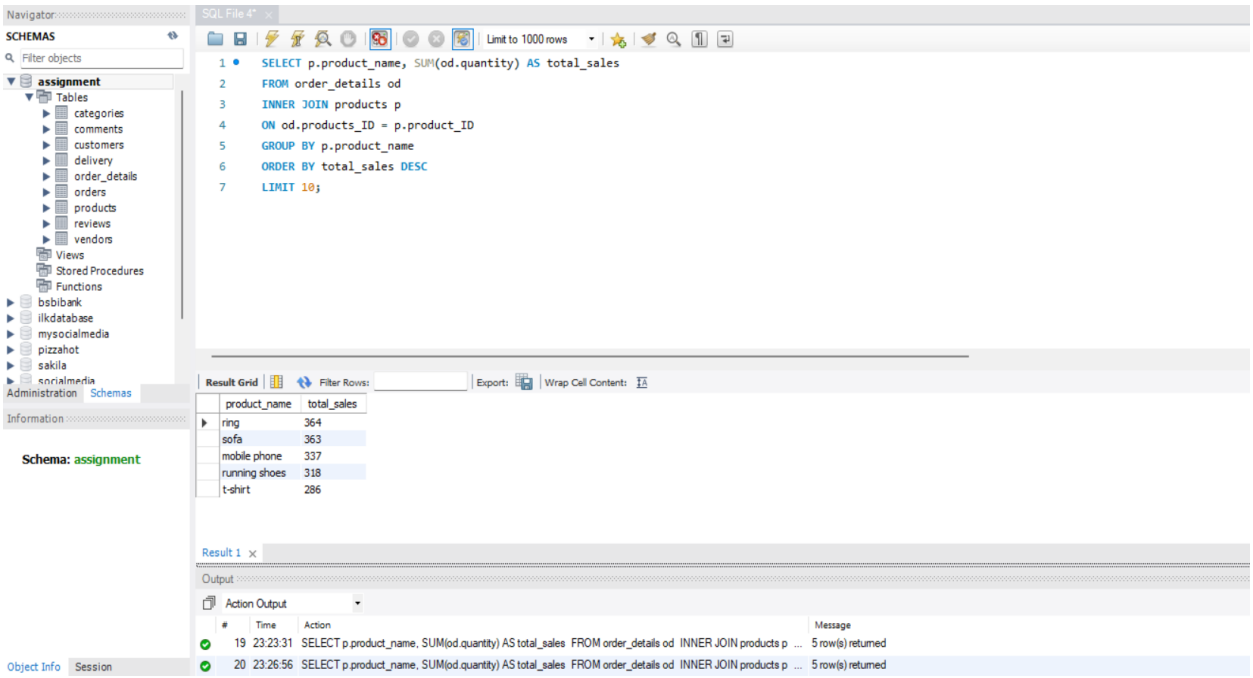

Figure 20 – Best – selling products

To find the total revenue generated per product by category type over the past year, I wrote an SQL query, which is shown in figure 21.

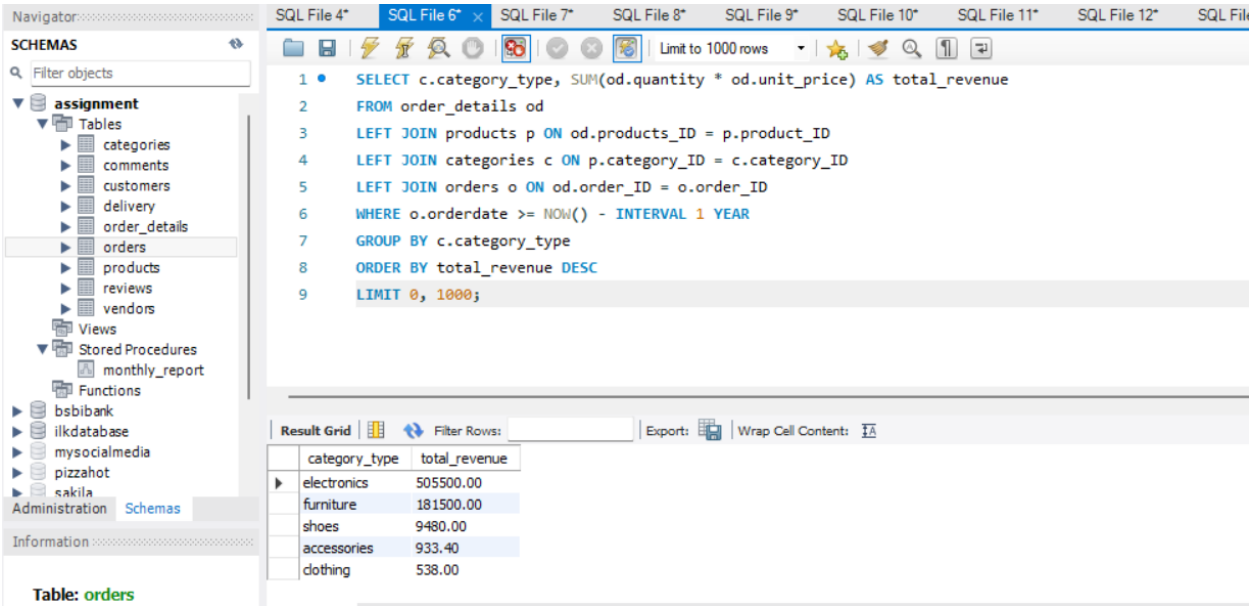

Figure 21 - Total revenue generated per product by category type over the past one year

To find the average revenue for every category name, I wrote an SQL query, which is shown in figure 22.

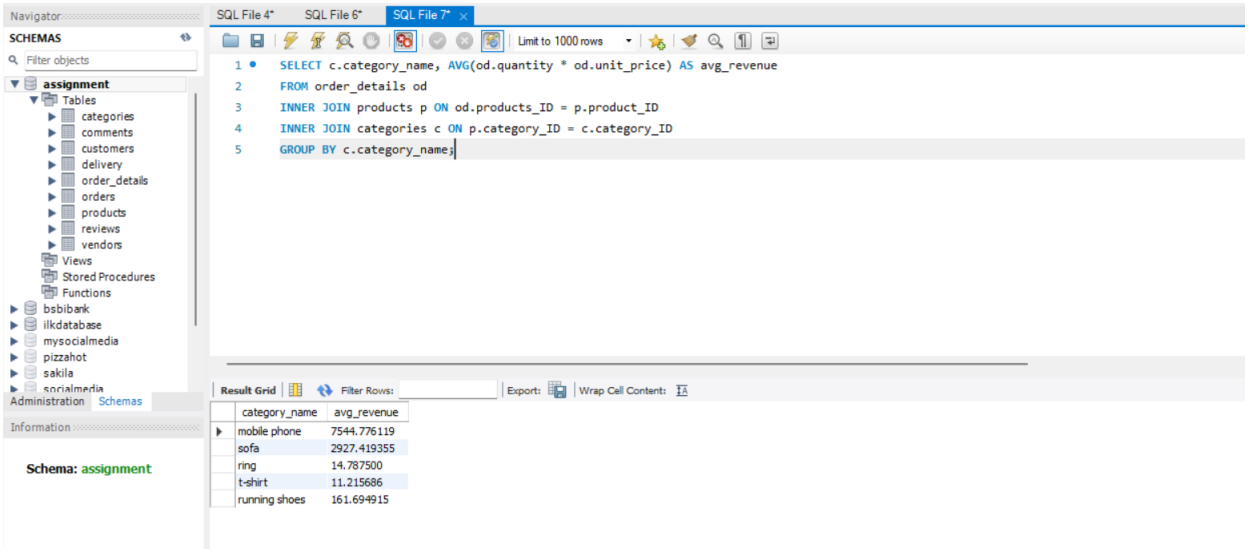

Figure 22 - Average revenue for every category name

For sub-task 1.1, I focused on customers' purchasing behaviors and wrote the following SQL query to answer this question, as shown in figure 23.

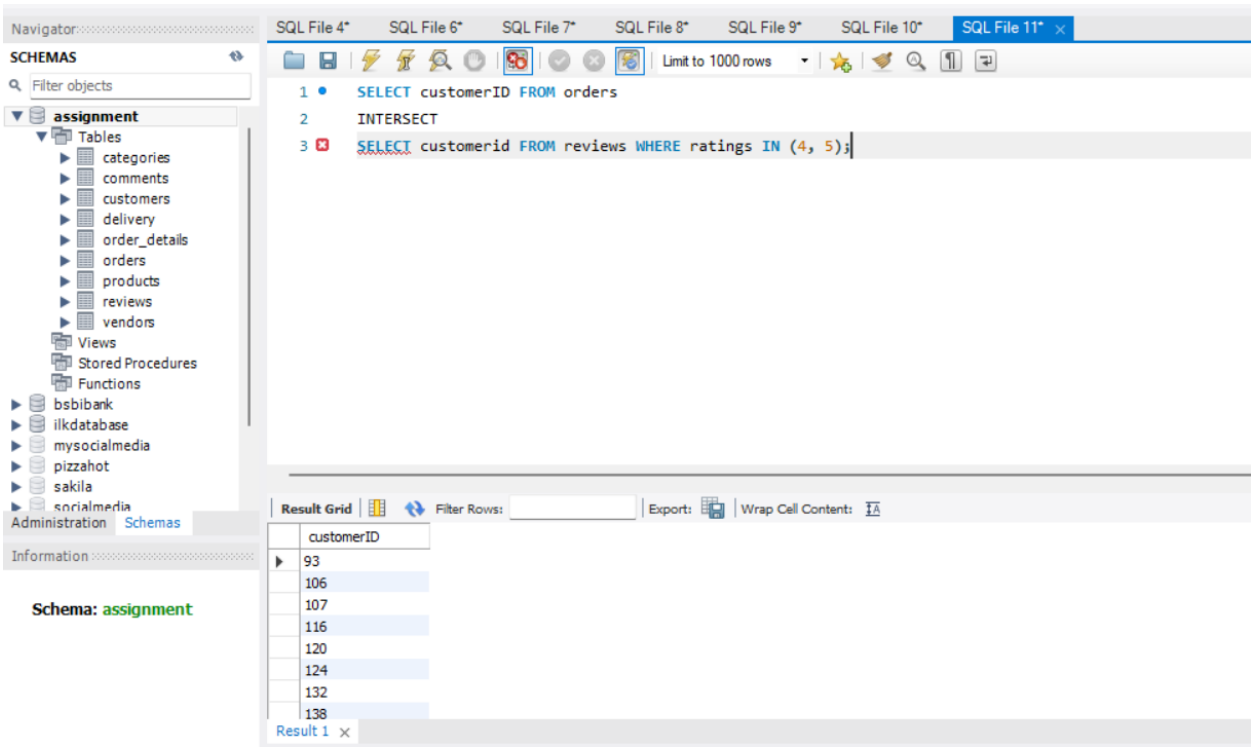

Figure 23 - Customers who have placed orders and write reviews with ratings of 4 or 5

According to this query, I can find customers who ordered products and wrote reviews with high ratings on the other hand, customers who ordered products and satisfied with this experience.

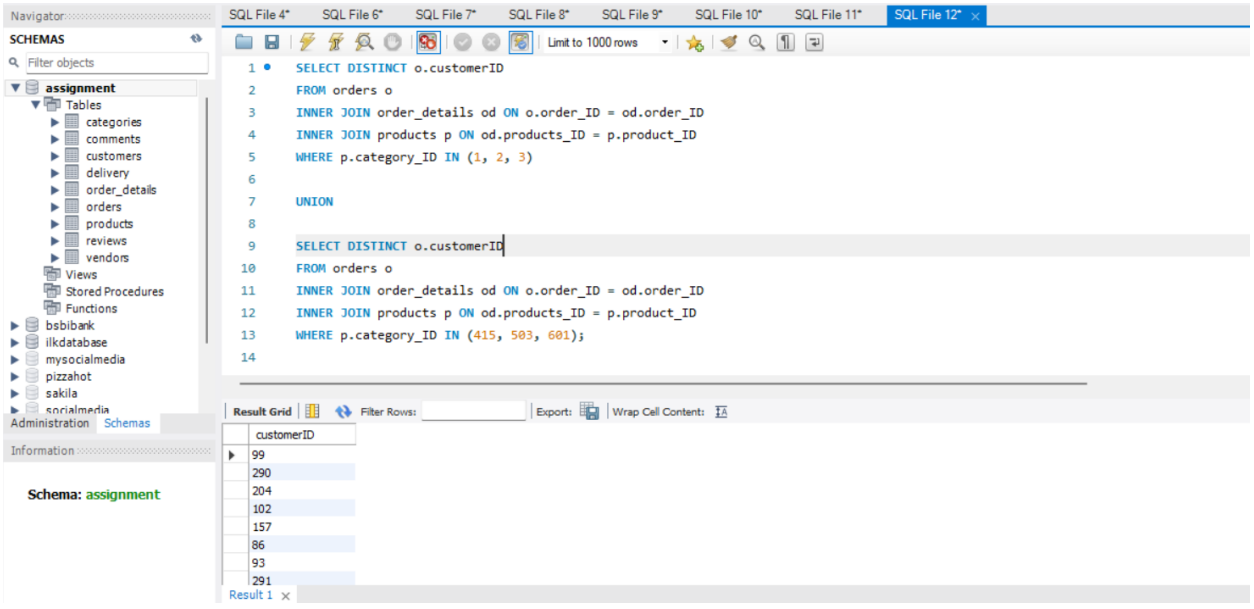

Figure 24 – Customers who have placed orders in category ID 1-2-3 and also 415-503-601

According to this query, I can find customers who ordered products in specific categories. I had chosen these categories randomly.

For sub-task 1.1, I focused on whether customer's age affects their review in a good or bad way and wrote the following SQL query to answer this question, as shown in figure 25.

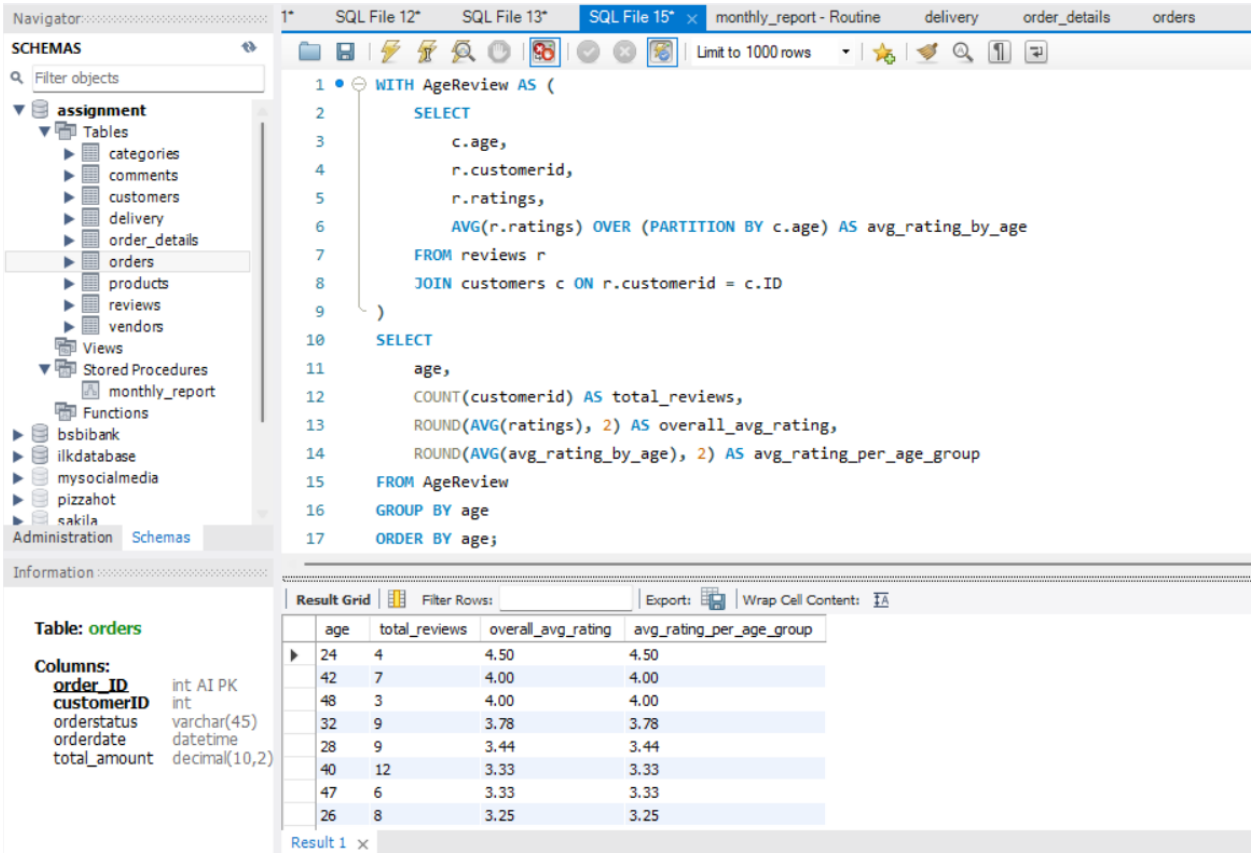

Figure 25 - Customer's age affects their review

According to this query, I can determine whether younger or older customers give higher ratings. For example, customers aged 24 give higher ratings, while customers aged 34 give the lowest ratings.

In sub-task 2.1, I mentioned comment date corrections as shown in figure 11. This correction is also called 'Trigger function'.

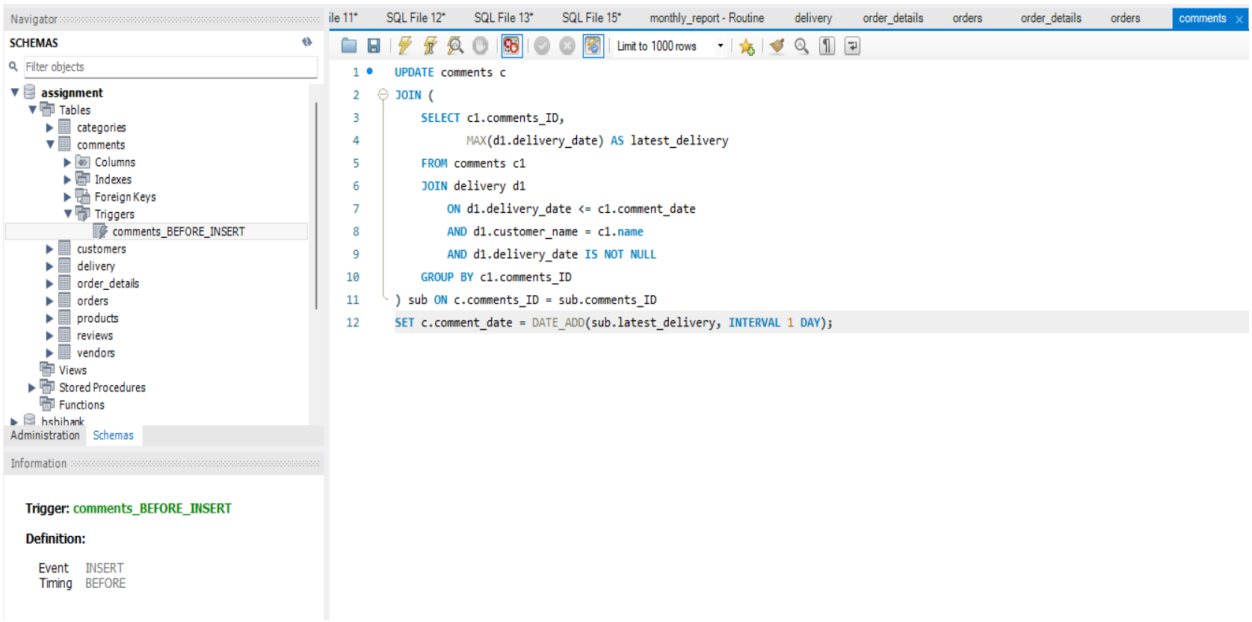

Figure 26 – Trigger Function

This trigger function adjusts the comment date is at least one day after the delivery date. One of the most remarkable points is 'delivery date is not null' query. With these sentences my trigger function ignore delivery date = null which means orders not delivered yet.

SUB-TASK 2.3

Stored procedures are crucial for the SQL database system. They improve efficiency and security by reducing redundancy, ensuring consistency and improving availability. In traditional SQL query methods, calculating monthly sales or calculating monthly revenue may require multiple repeated queries and redundant code. However, stored procedures eliminate code duplication and reduce the redundancy.

Without stored procedures, Executing complex queries increases the processing time and system working load. If too many queries are executed at the same time, database performance can decrease. Instead of running inefficient queries, the stored procedures handle requests efficiently.

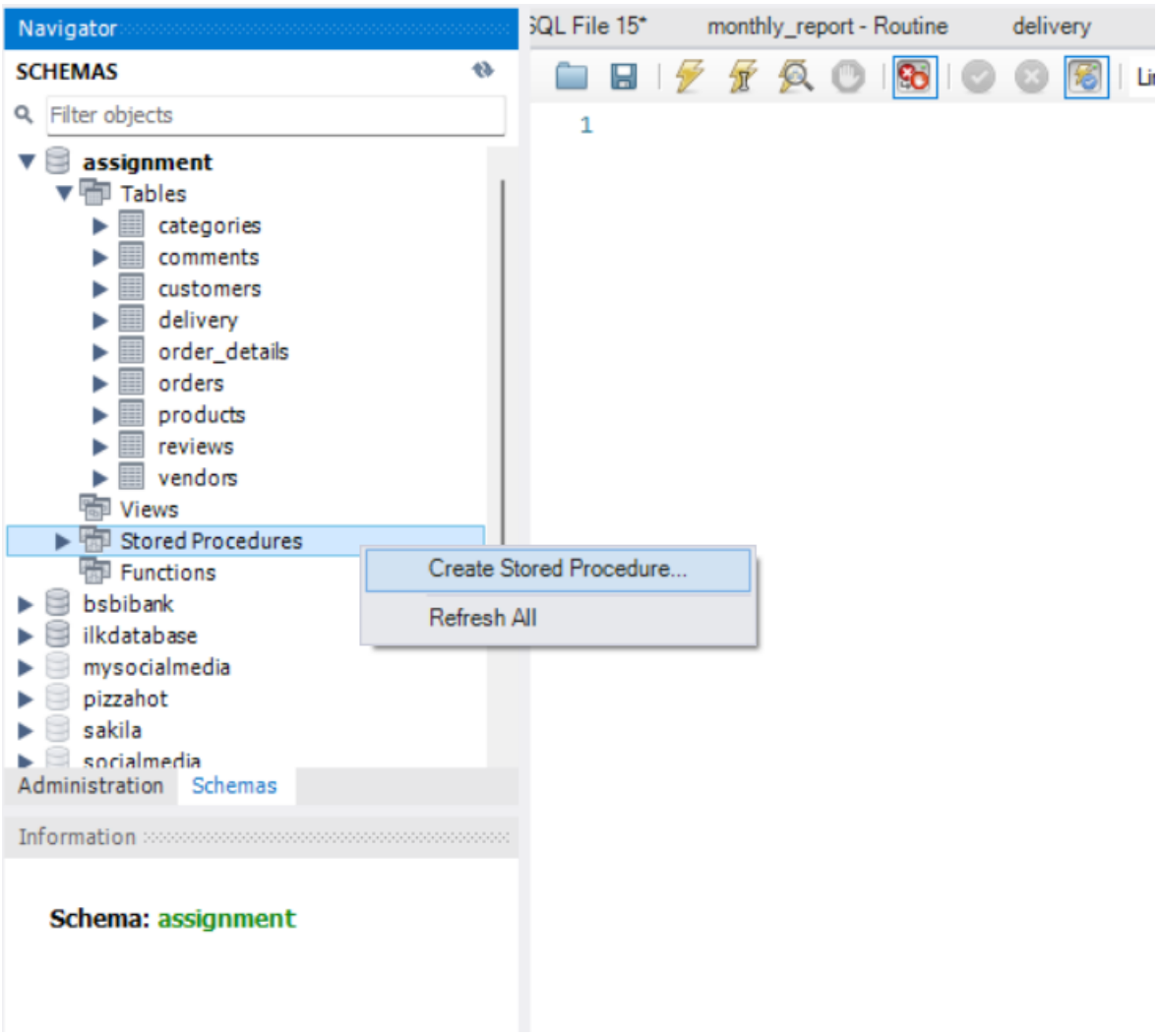

Figure 27 – Creating Stored Procedure

To create a stored procedure, we have to right-click on the navigator menu at the left side of the screen. When we click on the 'Create Stored Procedure', a new screen is shown below in figure 28.

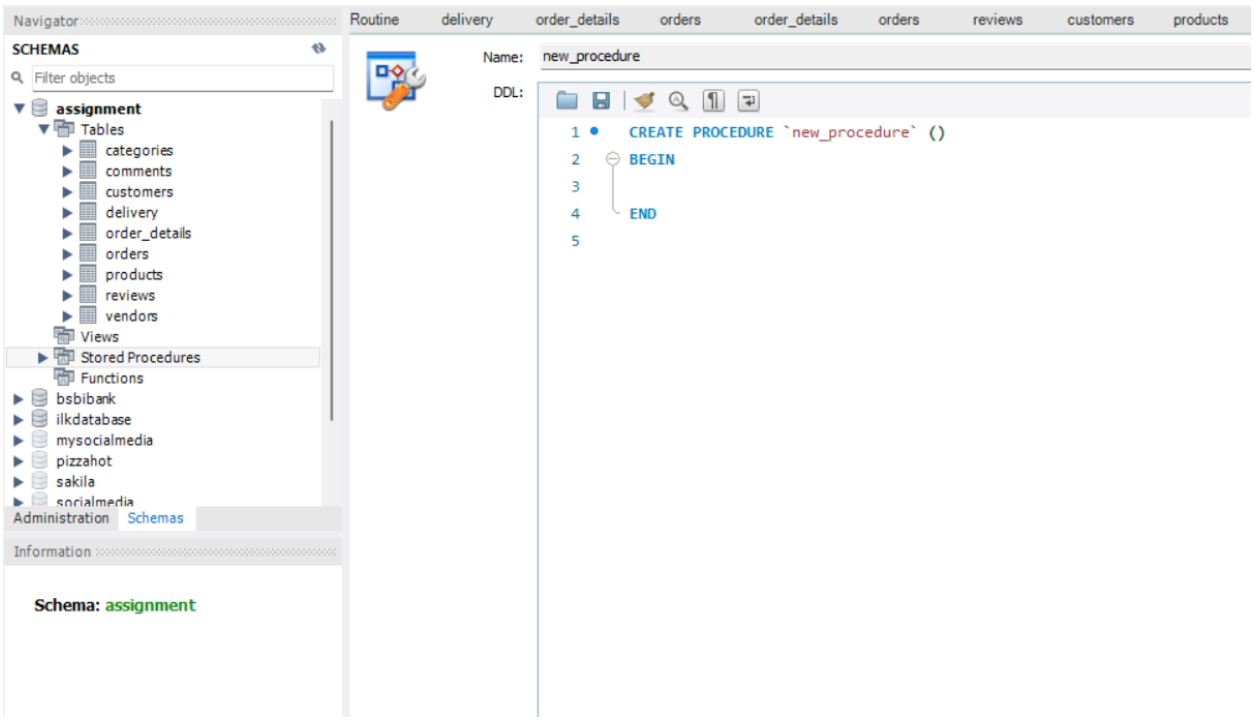

Figure 28 – Creating Stored Procedure 2

At this stage, we have to write the name of the stored procedure at the name line and also in the first query line instead of 'new procedure'. In figures 29 and 30, the necessary SQL queries to create the stored procedure named 'monthly report' are shown.

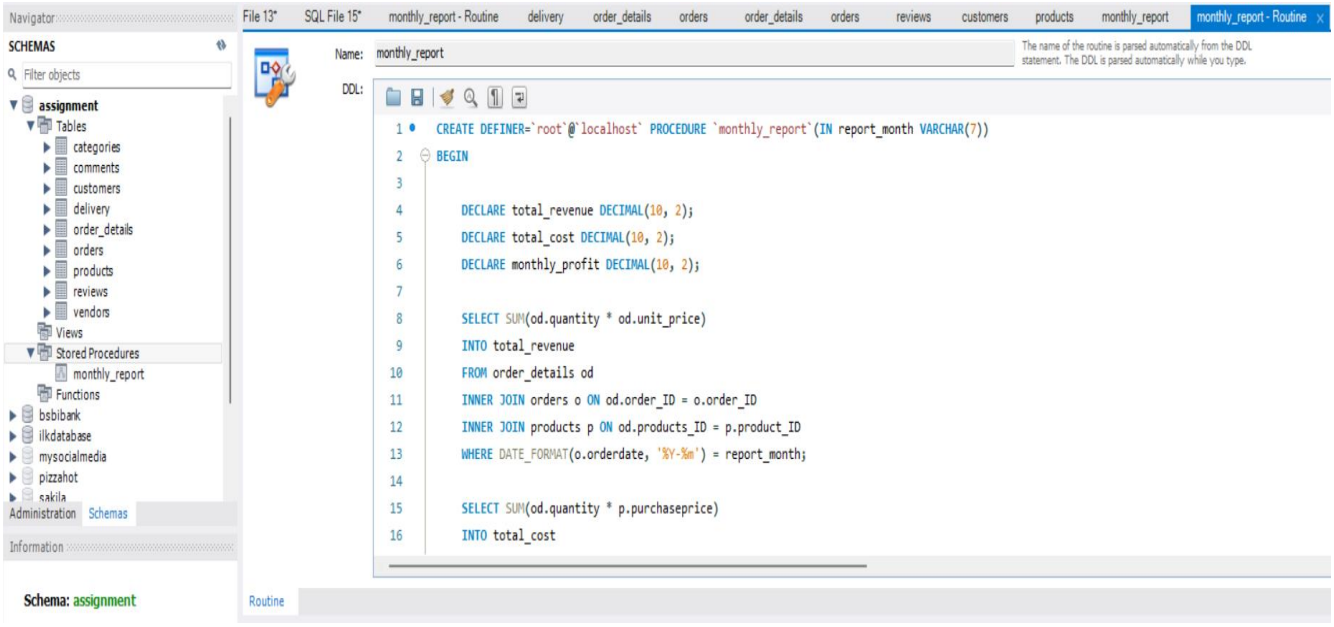

Figure 29 – SQL Queries for creating stored procedure named 'monthly report' 1

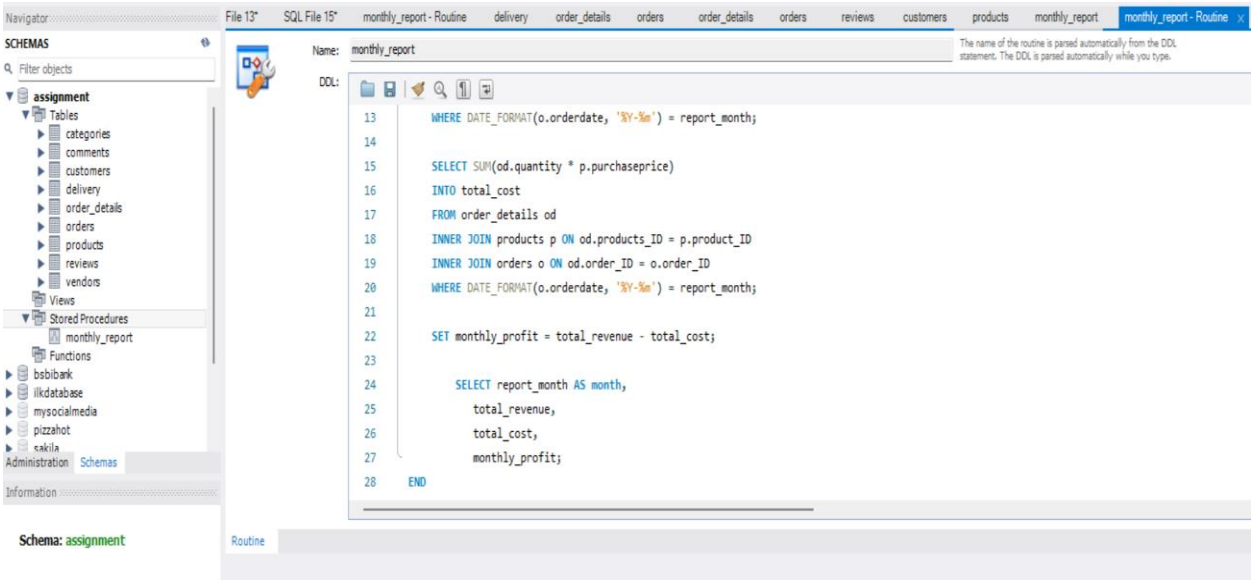

Figure 30 - SQL Queries for creating stored procedure named 'monthly report' 2

After running the queries, our stored procedure are created and ready to work. When I click small thunder icon which is next to monthly_report sentence, stored procedure function shown in figure 31.

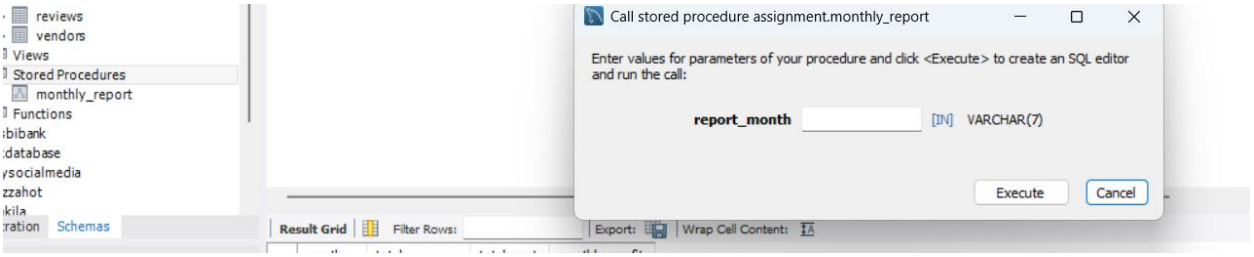

Figure 31 – Stored Procedure function

When I randomly write '2024-05', and click execute the outcomes screen as shown in figure 32.

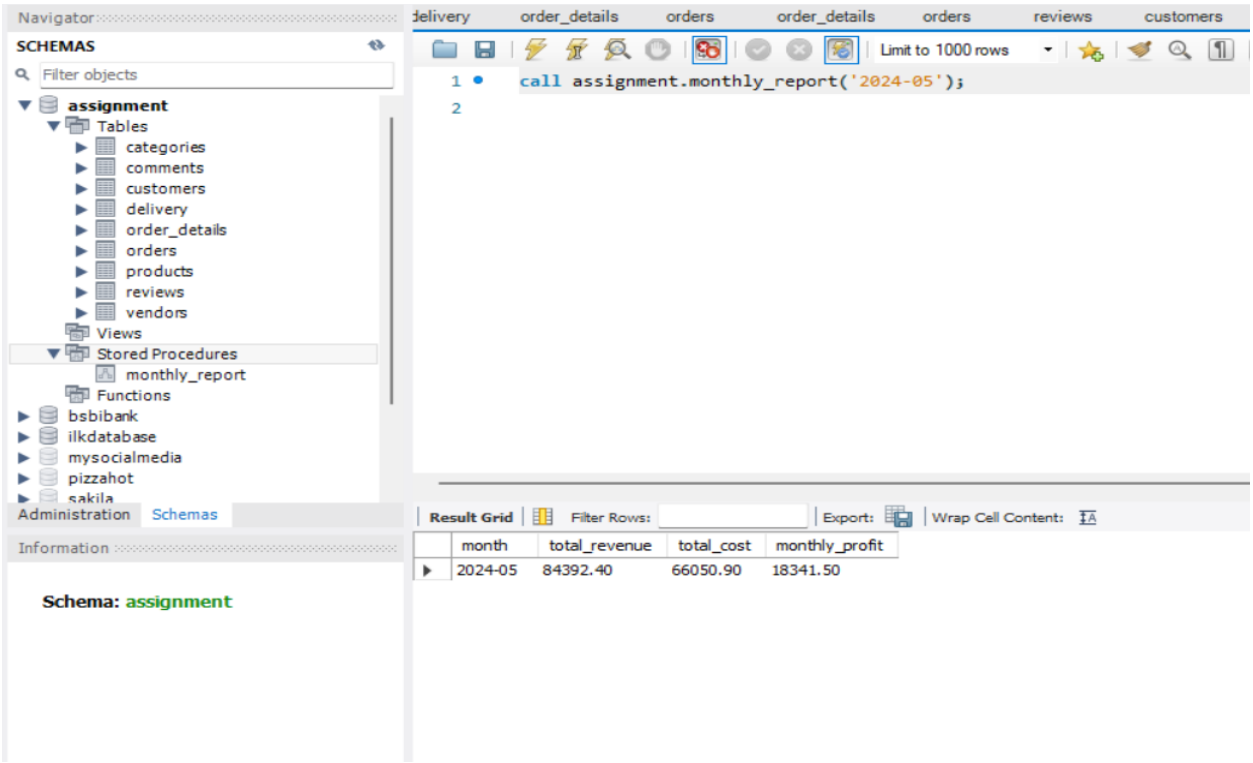

Figure 32 – Outcomes with Stored Procedure

CHAPTER THREE

TASK 3 – THEORETICAL DISCUSSION

SUB – TASK 2.4

Without making any changes in our data warehouse, we can analyze customer purchasing behavior and vendor value. Based on customer reviews and ratings, we can track product quality and vendor sales performance and as a result of this analysis, we can terminate our contract with vendors who sales poor quality products.

SUB – TASK 3.1

To analyze the query, I selected the window function which is shown at figure 25. With this query, we can calculate the average review ratings per customer age. This query is more efficient than using multiple joins. 'WITH AgeReview As ()…' query minimizes data duplication.

If the dataset were 10x bigger than now, the performance of the query could decrease due to more data being processed in the JOIN between customers and reviews. Naturally this would cause more memory usage and affect the scability and efficiency.

SUB – TASK 3.2

A distributed database is a collection of multiple, logically interrelated databases distributed over a computer network. Like the central database system, it is necessary to ensure that the data in a distributed database maintains integrity so that all users who access the database can retrieve the correct data and run their programs without errors. To address this issue, the ACID properties can be applied to distributed databases. These properties—Atomicity, Consistency, Isolation, and Durability—ensure the reliability of database transactions (Yu, 2008).

Atomicity: In our online marketplace, when a customer proceeds to checkout the system must ensure that the order is created and the payment is deducted from the customer's balance. If any steps fails or an error occur, the entire transaction must be rolled back.

Consistency: In our online marketplace, if a customer accidentally clicks the "Buy Now" button twice and sends two identical orders, we enforce a UNIQUE constraint to prevent duplicate records. If the customer submits the same order twice within a short time, the second attempt fails.

Isolation: In our online marketplace, if two customers try to buy the same products from the same vendors at the same time, MySQL uses transaction isolation levels to prevent race conditions. This prevents both customers from buying the same item.

Durability: In our marketplace, if a customer successfully makes a payment, MySQL ensures that the payment is not lost even if the system crashes. If the server crashes, MySQL's transaction logs guarantee that the payment is not lost.

The ACID properties ensure that marketplace transactions (such as purchases, payments, and inventory updates) are reliable, preventing issues like double payments, lost data, and inconsistencies. These guarantees are crucial for smooth operations in e-commerce platforms.

SUB – TASK 3.3

The CAP theorem is also known as Brewer's Theorem, because it was introduced by MIT Professor Eric A. Brewer during 2000 with the concept of distributed computing. Our main purpose in this paper is to consider the influence of CAP Theorem in the broader perspective of big data analysis and distributed computing theory. (Pandey, Anand Kumar, and Rashmi Pandey, 2020)

The CAP theorem explains the thought that there is an elementary transaction between availability, consistency, and partition tolerance. This transaction, which has become identified as the CAP Theorem, has been commonly discussed ever since. (Pandey, Anand Kumar, and Rashmi Pandey, 2020)

CAP Theorem is considered that a distributed database organization can only consist of 2 of the 3 properties: Consistency, Availability, and Partition Tolerance. (Pandey, Anand Kumar, and Rashmi Pandey, 2020)

Generally, consistency is crucial because the users want to access the correct and current data. For instance, when a customer makes a payment and places an order, all relevant data, such as payment status, order details, and stock level in the system, have to be consistent to ensure that the data is accurate and integrated.

Our data warehouse is designed with CA supported model which means supports and guarantees consistency and availability. In critical processes such as transactions and orders, serious consequences could happen if there are any inconsistencies, and as a result of inconsistency, the customer's experience can be negatively affected and the results can be very serious and severe.

For example, if an error occurs, while a customer is placing an order or during the transaction, the system must protect itself and with a rollback mechanism, all processes must be rolled back.

Our data warehouse must support high availability because of customer satisfaction. For example, if a customer requests to view their order details while visiting our e-commerce website, the data warehouse must remain fully operational and available without any delay or connection error. This is crucial because interruptions could lead customers to cancel their orders.

By choosing the CA model instead of CP or AP in our data warehouse, we ensure that users always receive consistent and available data, which is crucial for business and customer satisfaction. Furthermore, the CA model enhances the capability of keeping reports consistent, allowing the

customer and the vendor to rely on the system with confidence. By having high availability, the CA model can reduce operational interference.

However, while Partition Tolerance (P) is considered of lesser priority by us, our system is built with strategies to mitigate any network partition failure. These measures help maintain a balance between availability and consistency while minimizing the impact of potential system failures.

CONCLUDING REMARKS

This report shows the importance of designing and effectively managing an analytics data warehouse for a global e-commerce platform. The successful implementation of the data warehouse design enables the platform to better understand customer behaviors, sales trends, and vendor performance.

The data warehouse allows for the fast and accurate analysis of data, providing valuable insights for business management. Particularly, it has big benefits in optimizing customer satisfaction, product recommendations, and sales strategies.

In this data warehouse, I focused on simplicity and efficiency. I tried to create simple but useful tables and data warehouse and aimed for the efficient usage of all users.

BIBLIOGRAPHY

REFERENCES

1 - Yu, S. (2009) 'ACID in Distributed Databases', *Advanced eBusiness Transactions for B2B-Collaborations*, [pdf] Available at: https://www.cs.helsinki.fi/group/cinco/teaching/2009/advanced-businesstransactions-seminar/papers/ACID_in_Distributed_Database_Shiwei_Yu.pdf (Accessed: 11 February 2025).

2 - Pandey, Anand Kumar, and Rashmi Pandey. "Influence of CAP Theorem on Big Data Analysis." *International Journal of Information Technology (IJIT)*, vol. 6, no. 6, 2020. (Accessed 11 Feb. 2025).

APPENDIX

- ✓ https://colab.research.google.com/drive/104gu_by8NTKvdQhr9quKPoRHkkE83mAg#scrollTo=rKD-ENWg2c3H
- ✓ https://miro.com/app/board/uXjVLoaLmb4=/
- ✓ https://github.com/sefakpt/enterprise-assignment